Parameters and Hyperparameters

Hyperparameters: Introduction

What are parameters and hyperparameters?

- *W, b*
- Learning rate
- Number of hidden layers
- Number of hidden units
- Activation functions
- Momentum, mini-batch size
- Regularization

Guidelines

Disclaimer: **no one-size-fits-all recipe exists**

Learning Rate

- The learning rate **controls the step size of weight updates** during gradient descent.
- A smaller learning rate leads to slower convergence but may result in better final performance.
- A larger learning rate may cause faster convergence but could overshoot the optimal solution or cause instability.
- **Advice:** Consider using learning rate **scheduling techniques** to adapt the learning rate during training.

Number of Hidden Layers

- Adding more hidden layers allows the network to learn more complex, hierarchical representations of the input data.
- However, increasing the number of hidden layers may result in overfitting and longer training times.
- **Advice:** Start with a small number of hidden layers and increase gradually, monitoring performance on a validation set.

Number of Hidden Units

- The number of hidden units in each layer **affects the capacity** of the model to learn complex patterns.
- Too few hidden units may limit the model’s ability to learn, while too many may cause overfitting.
- **Advice:** Experiment with **different numbers** of hidden units, considering the complexity of the problem and the size of the input data.

Activation Functions

- The choice of activation function can have a significant impact on the performance and convergence of the model.
- Some activation functions have parameters to tune.
- ReLU is a popular choice for hidden layers due to its simplicity and ability to mitigate the vanishing gradient problem.
- For binary classification tasks, consider using a sigmoid activation function for the output layer.

Momentum and Mini-Batch Size

- Momentum can help accelerate training and overcome local minima in the optimization landscape.
- Mini-batch size affects the trade-off between computational efficiency and convergence stability.
- **Advice:** Experiment with different momentum values and mini-batch sizes to find the best combination for your specific problem.

The End

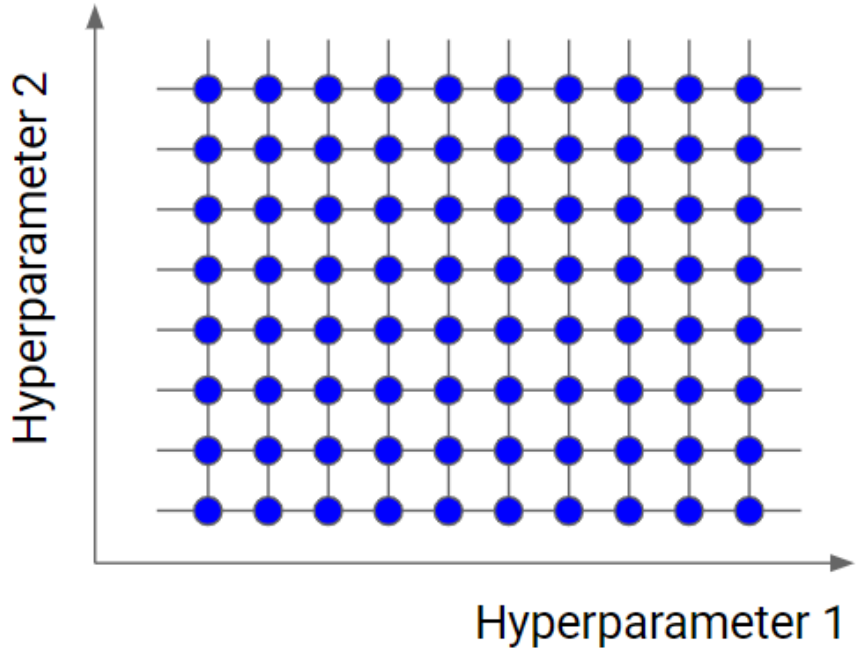
Systematic Approaches for Tuning Neural Network Hyperparameters

Introduction

- Hyperparameter selection is a crucial step in building a high-performing neural network.
- Different strategies can be employed to find the optimal combination of hyperparameters.
- This presentation will cover three popular strategies: grid search, random search, and Bayesian optimization.

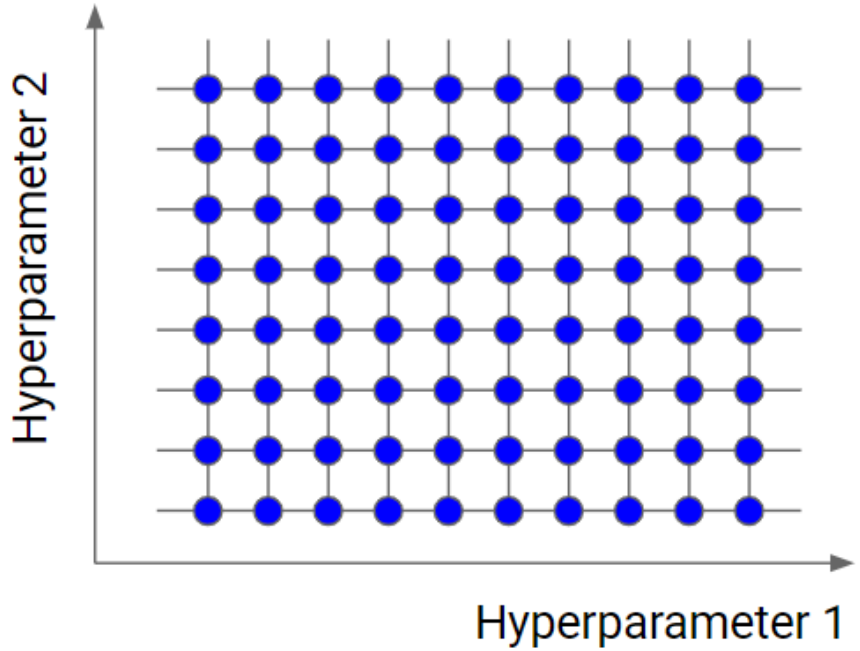
Grid Search Overview

- Grid search is a simple and **exhaustive search** technique
- Useful for hyperparameter tuning
- Explore **all possible combinations** of hyperparameter values in a predefined search space



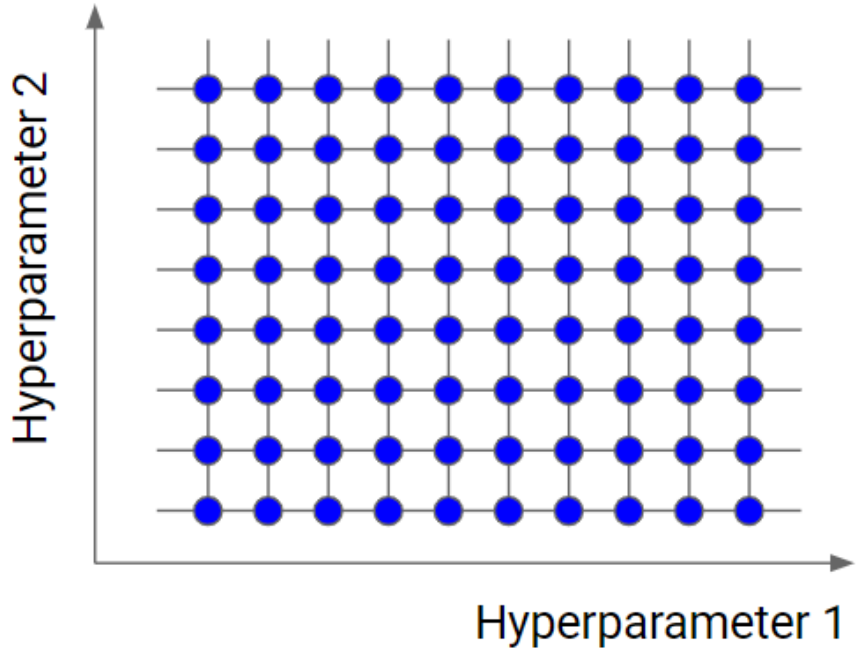
Grid Search Procedure

1. Define a search space for each hyperparameter.
2. Create a grid of all possible hyperparameter combinations.
3. Train and evaluate a model for each combination on a validation set.
4. Select the combination that results in the best performance.



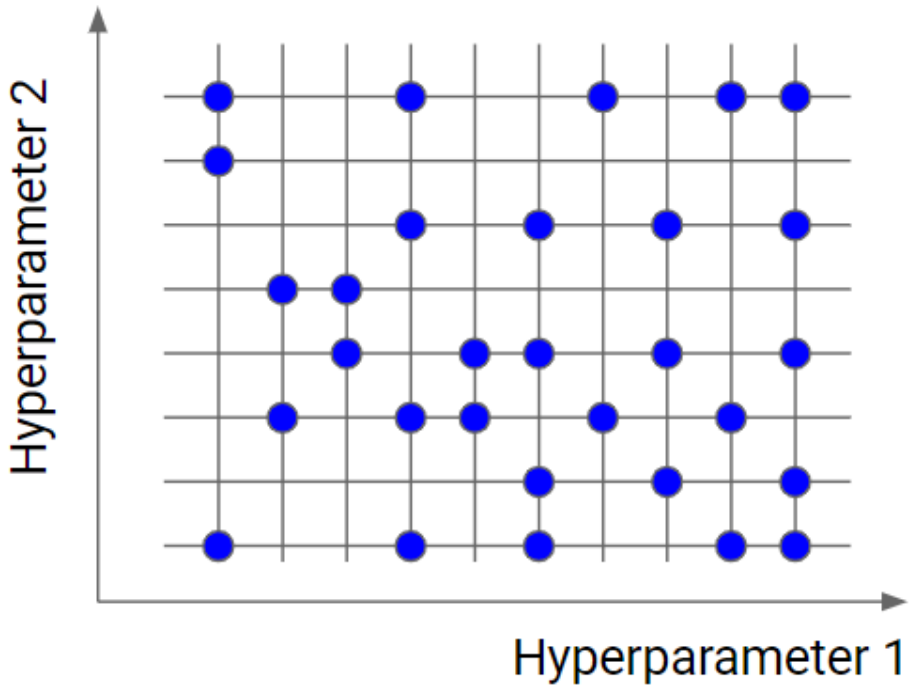
Grid Search Pros and Cons

- **Easy to implement**, guarantees finding the best combination within the specified search space
- **Computationally expensive**, especially for large search spaces
- **Not efficient for continuous** hyperparameters



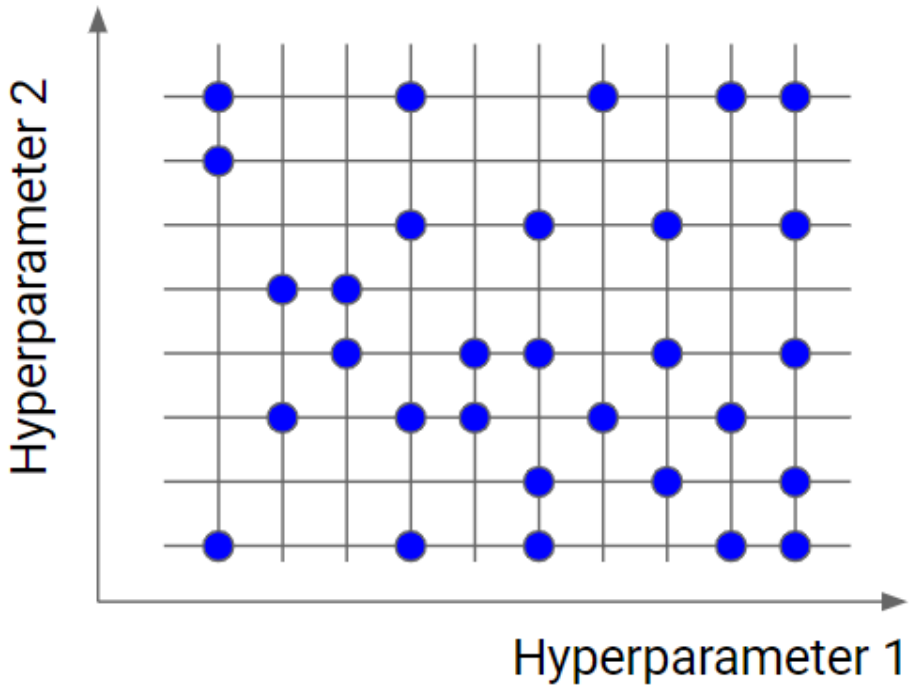
Random Search Overview

- Random search is an alternative to grid search that explores the search space **more efficiently**.
- It involves **sampling** hyperparameter values **randomly** from a predefined distribution.



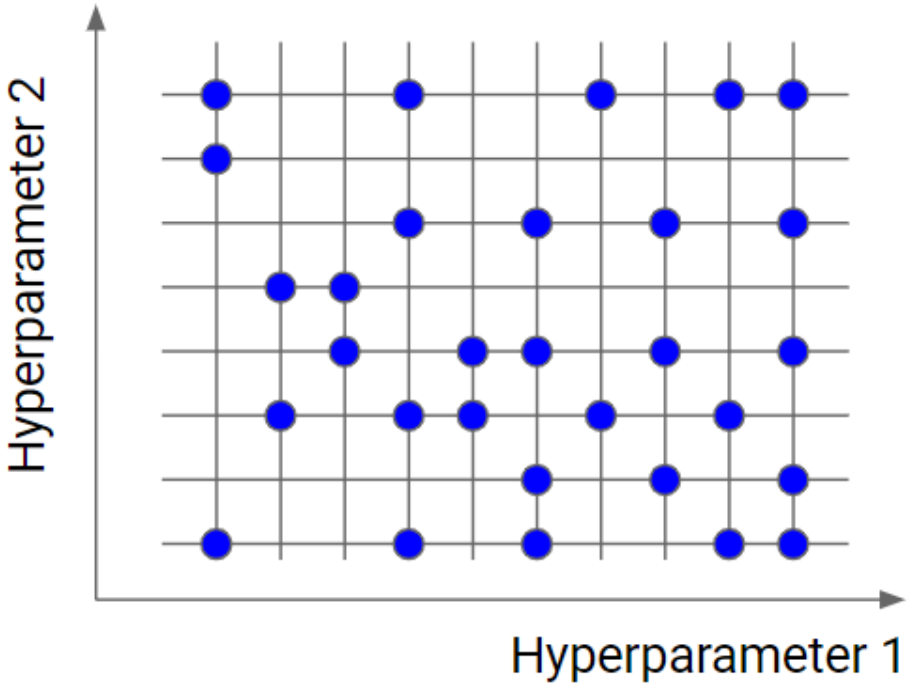
Random Search Procedure

1. Define a search space and probability distribution for each hyperparameter.
2. Randomly sample hyperparameter values.
3. Train and evaluate a model for each sampled combination.
4. Repeat the process until a stopping criterion is met.



Random Search Pros and Cons

- More efficient than grid search (for large search spaces)
- Can find good combinations with fewer evaluations
- No guarantee of finding the best combination
- May require more iterations for high-dimensional hyperparameters



Bayesian Optimization Overview

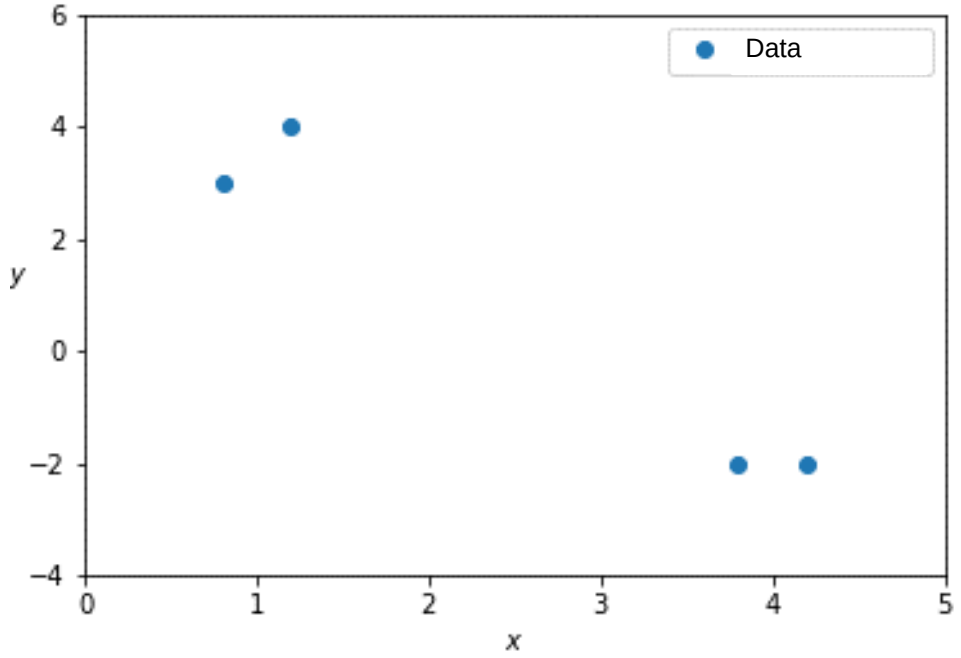
- Bayesian optimization is an advanced, probabilistic technique for hyperparameter tuning.
- **Main idea:** Use prior knowledge (previous evaluations) to guide the search for the optimal hyperparameters in an intelligent and efficient manner.

Bayesian Optimization Procedure, Part I

1. **Define a search space** for the hyperparameters you want to optimize. For example, you may want to find the best learning rate and number of hidden units for your neural network.

Bayesian Optimization Procedure, Part II

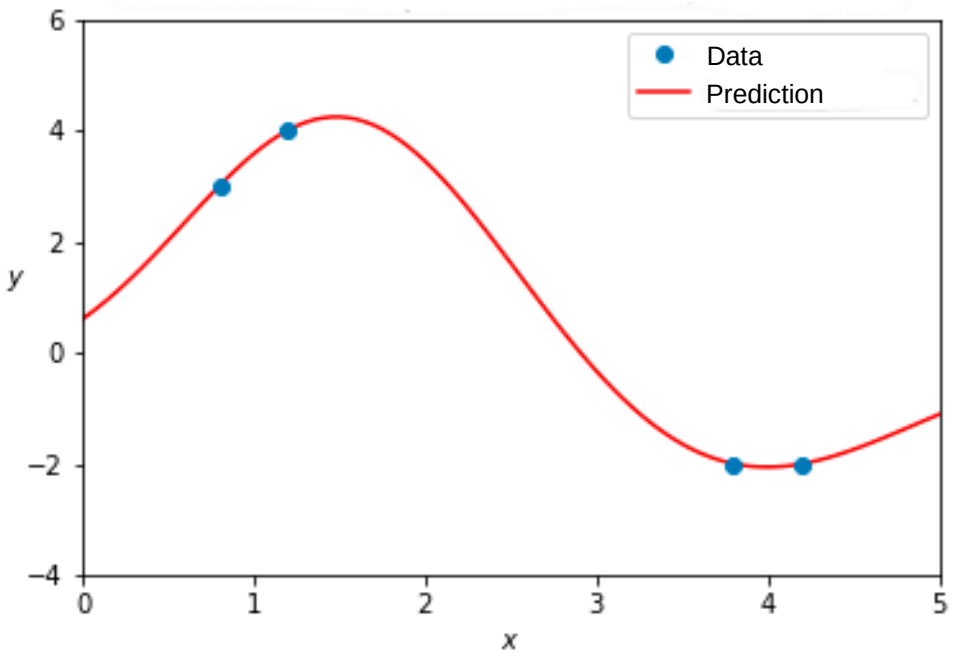
2. **Evaluate the performance** of your model on a few random combinations of hyperparameters. This initial evaluation will give you some information about how well your model performs with different hyperparameter settings.



Model performance y for random values of hyperparameter x

Bayesian Optimization Procedure, Part III

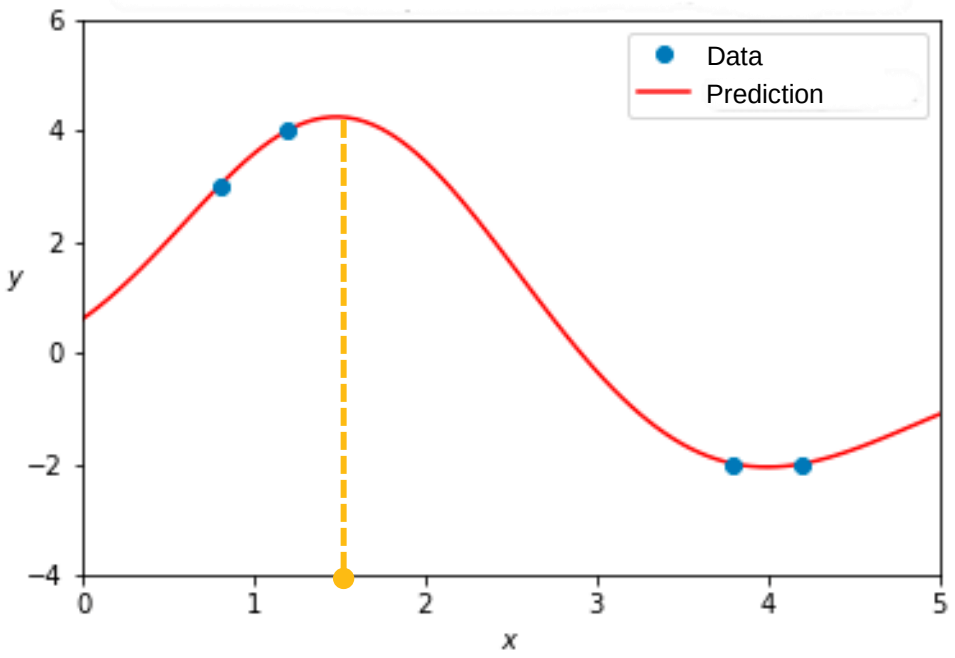
3. Now, instead of randomly choosing the next combination of hyperparameters to evaluate, **use a probabilistic model** (called a surrogate model) to estimate the performance of all possible combinations based on the previous evaluations.



Model performance y for random values of hyperparameter x

Bayesian Optimization Procedure, Part IV

4. **Select the next combination of hyperparameters** to evaluate based on the predictions of the surrogate model (usually, the combination with the best estimated performance is then selected).



Model performance y for random values of hyperparameter x

Bayesian Optimization Procedure, Part V

5. Evaluate the chosen combination of hyperparameters and update the surrogate model with the new performance information.
6. Repeat steps 4 and 5 until a stopping criterion is met (e.g., a maximum number of evaluations, no improvement for a certain number of iterations, etc.).

Bayesian Optimization Pros and Cons

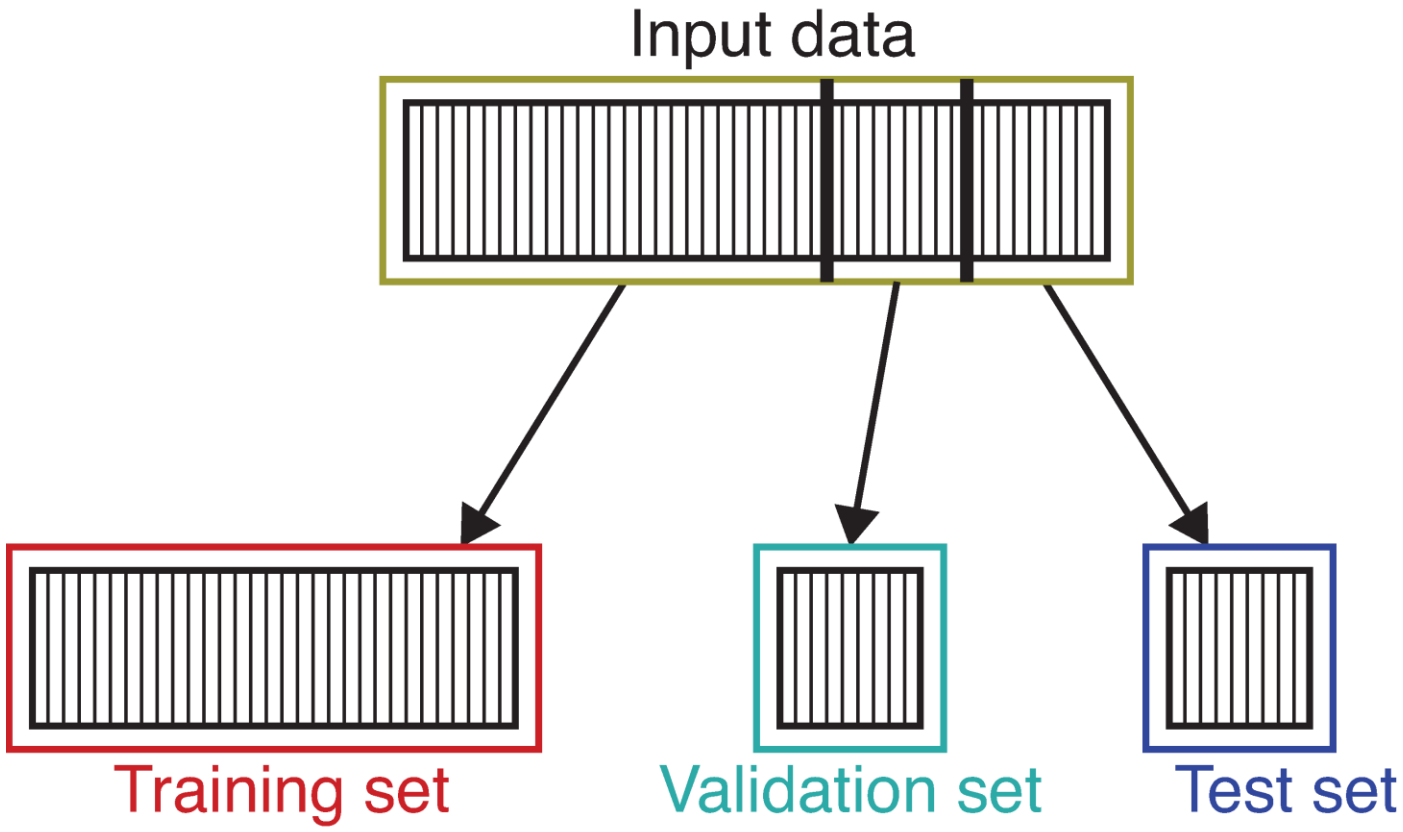
- More efficient than grid and random search
- Good for high-dimensional or continuous hyperparameters
- Adaptively focus on promising regions of the search space
- More complex to implement
- Can be sensitive to the choice of surrogate model and acquisition function

The End

Training/Validation/Test Sets

Training/Validation/Test Sets

How to break data in training, validation, and test sets?

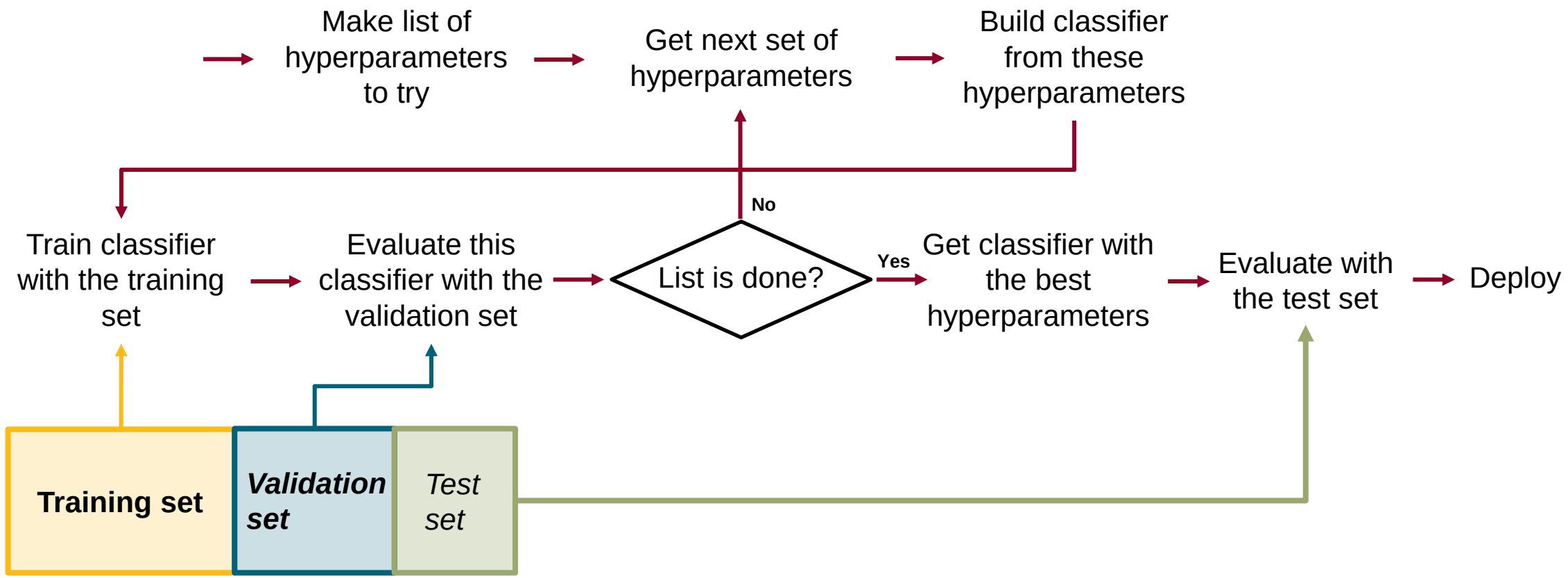


Creating New Datasets

- Make sure the train data is representative of the real-world scenario it will encounter

- Make sure the validation/test data come from same distribution as training

Workflow and Dataflow



What to Do

High bias

- Bigger NN
- Alternate architecture
- Train longer

High variance

- Bigger NN
- Alternate architecture
- Regularization techniques

The End

Normalizing Input

Normalizing Input

- Features span very different numerical ranges. Example: African bush elephants
  - Age in hours (0 to 420,000)
  - Weight in tons (0 to 7)
  - Tail length in centimeters (120 to 155)
  - Age relative to mean age in ours (-210,000 to 210,000)

- Normalize each feature (i.e. [-1,1] or [0,1]).
- Standardization

The End

Vanishing / Exploding Gradients

Vanishing / Exploding Gradients

- **Vanishing Gradients:** The gradients become very small, slowing down learning and making it difficult for the model to update its weights.
- **Exploding Gradients:** The gradients become excessively large, causing unstable updates and potentially divergent training.

Weight Initialization

- Carefully initializing the network weights
- Common weight initialization techniques:
  - Uniform initialization
  - Normal initialization
  - LeCun Uniform/Normal initialization
  - Xavier/Glorot Uniform/Normal initialization

The End

Checking Gradients During Training

Checking Gradients During Training

- Monitoring gradients during training:
  - Identify vanishing or exploding gradients
  - Assess learning rate
  - Detect convergence

Checking Training and Validation Loss

- Monitoring losses during training:
  - Training loss
  - Validation loss
  - Comparing losses is important!

The End

Normalization

Batch Normalization

1. Compute the mean and variance of each feature in a mini-batch
2. Normalize the features by subtracting the mean and dividing by the standard deviation

Batch Normalization: Why It Works

- Prevents values coming out of the layers from drifting into very positive or very negative regions
- Prevents values from spreading out or compressing too much
- Keeps the values closer to the range where the activation function has its most useful non-linearities

Layer Normalization

- Similar to Batch Normalization
- Instead of normalizing across a mini-batch of examples, Layer Normalization normalizes across the features in each individual example
- Works well for RNNs and Transformer models

The End

Regularization

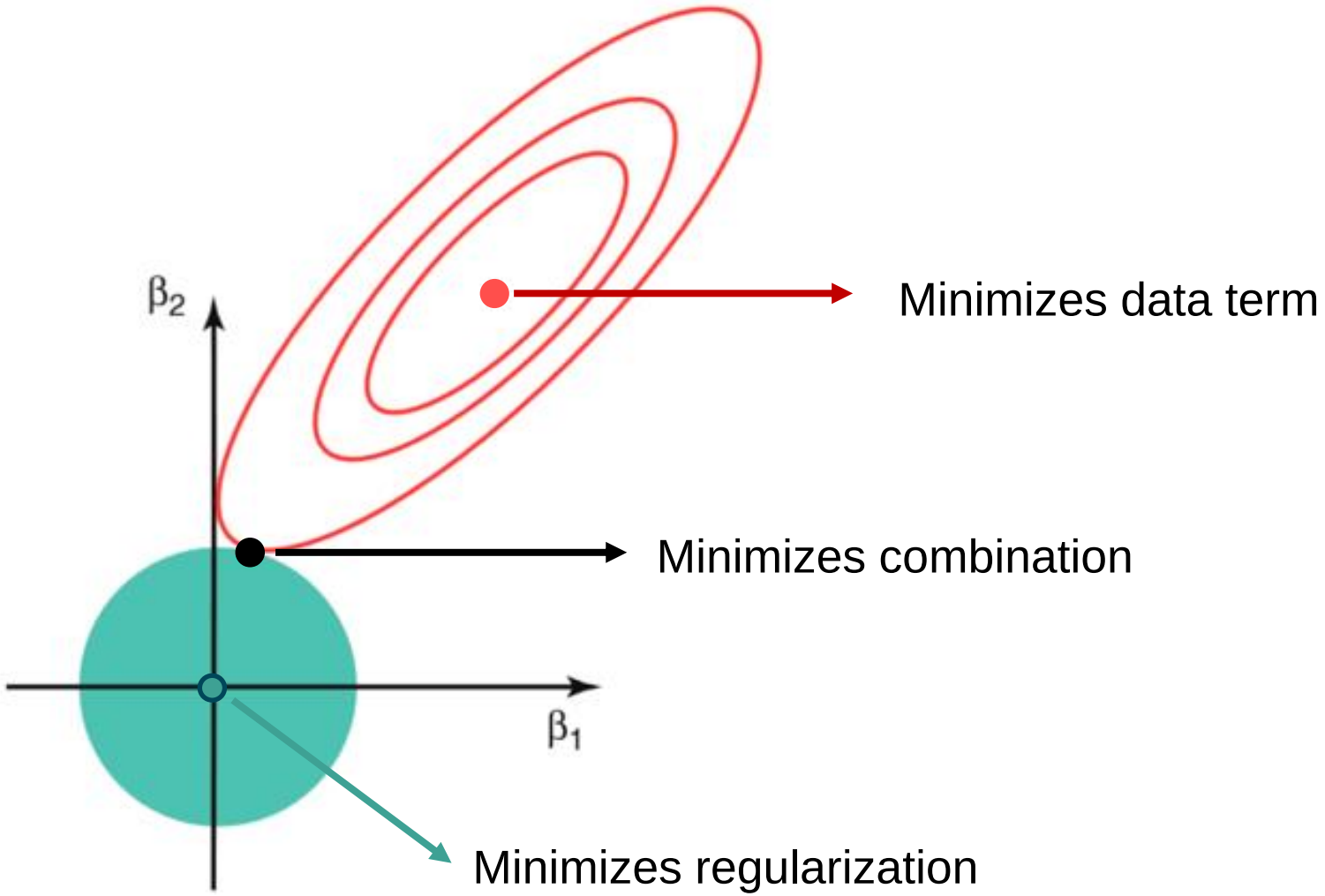
L1 Regularization

- Prevents overfitting by adding a penalty based on absolute values of weights
- Encourages smaller weights and **simpler models**
- Can force some weights to zero, “removing” certain connections
- Improves model's ability to generalize to new data
- Better overall performance

L2 Regularization

- Prevents overfitting by adding a penalty based on the square of weights
- Encourages **smoother decision boundaries**
- Spreads weight values more evenly across the network
- Improves model's ability to generalize to new data
- Better overall performance

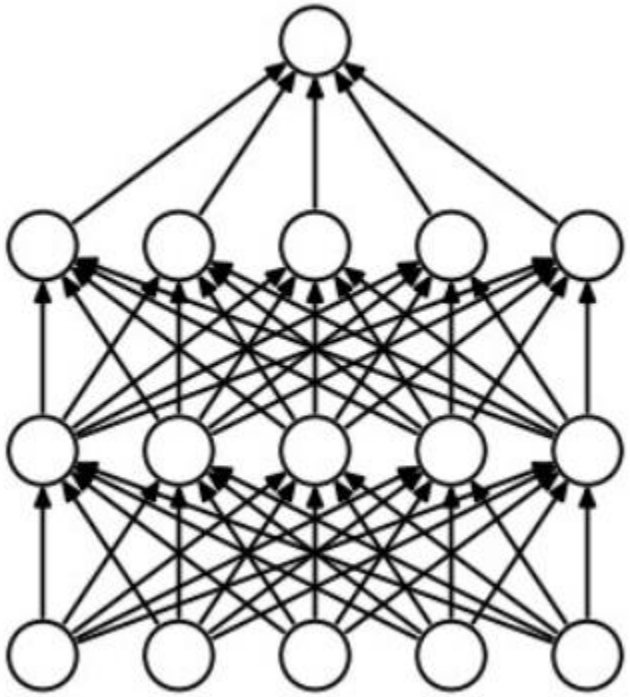
Visual Intuition



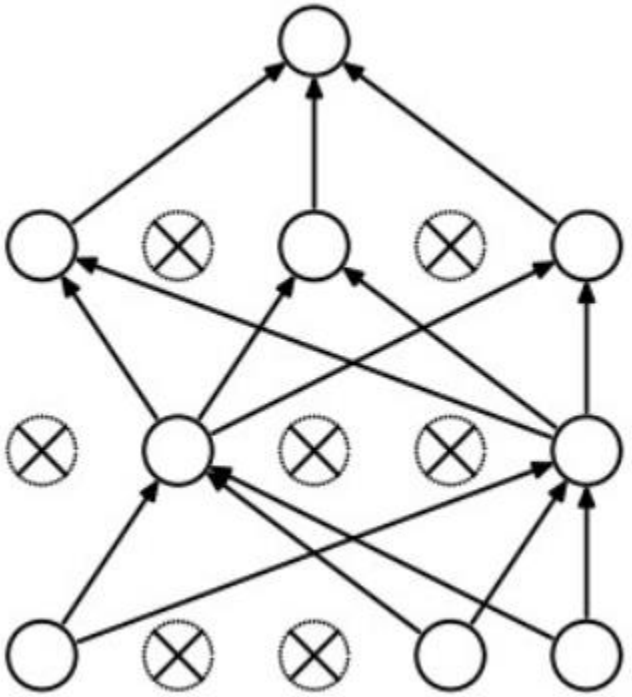
The End

Dropouts

Dropouts



Without Dropout



With Dropout

The End

Other Regularization Techniques

Other Regularizations

- Data Augmentation
- Noise Injection
- Early Stopping

The End

Assess the Problem

Where does problem originate

- Often through informal communications

AI process

Users

Business

Practitioners

A

Assess the problem

Is this a ML problem

- Business case + context
- Technical feasibility
- Business Impact?
- Formulate hypotheses
- sketch the solution(s)

B

Best Path to solve

Approach/metrics

- Evaluate models
- Design the solution
- Evaluation criteria
- Evaluate Metrics
- Ethical considerations
- Prioritize & Resources
- Time commitment

C

Configure Communicate

Communicate

- Technical feasibility
- Team.,
- Security
- Resources

D

Data acquisition exploration

Data get, explore

- Technical feasibility
- Acquire data
- Data wrangling, mangling, cleaning, filtering

E

Experiment deployment

Model, Evaluate

- Create training
- Evaluate the models
- Ethical considerations
- Communicate
- Productionalize

Assess the problem

- Understand the problem
  - Evaluate if it is ML problem
  - Which design
  - Understand the metrics, risks, ethical considerations

Strategies for troubleshooting

- Orthogonalization

The End

Build First System Quickly

Build First System Quickly

Options to create

- Data collection
- Data correction
- Getting the design right

Steel Thread Analogy

- Advantages
  - End-to-end framework is understood
  - Refining each subsystem is easy
  - Distribute subsystem refinement to new team member

The End

Data Distribution

Data distribution

- Training and test from different distribution
  - Training error / Dev error
  - Face recognition system Data
    - N. America
    - S. America
    - Europe
    - Middle East
    - Asia

Recommendation

- Shuffle data
- Check distributions of dev, test set
- Ensure your dev/test reflects real dataset
- Pick single metric

Data split

- Splitting data percentage Vs. Size
- K-splits

The End

Data Mismatch

Data mismatch

- Imbalanced data

Change the metric

- What to do when your test set is not from same as dev

- Redefine the error
- Establish measurements when test distribution changes
  - Aging, model revalidation
- Rebuilding
- Model warranty

The End

Error Analysis

Error Analysis

What is Error analysis

Definitions

- Bayes error
- Human-level performance

The End

Evaluation Metrics

Evaluation Metrics

- Introduction
- Rationale

Improving Performance

- Bayes error
- Human-level performance

The End

End-to-End Deep Learning

End-to-End Deep Learning

- Introduction
- Rationale

End-to-End Deep Learning (cont.)

- Examples

End-to-End Deep Learning (cont.)

- Examples

When to Use End-to-End Deep Learning

- Requirements
- How much data you have
- Do you know the interim structure?

The End